R3.09 - SAÉ3.01 – Walkthrough

Mot de passe

Explication pas à pas du TP

12 octobre 2026 – M.-B. Barry

Ce document reprend le sujet « R3.09 - SAÉ3.01 – Mot de passe » exercice par exercice. Pour chaque exercice : ce qui est demandé, le lien avec le cours « Fonctions de hachage – Stocker un mot de passe », le code et ce qu'on obtient en l'exécutant. Tout le code a été testé en Python 3.

Sommaire

1 Avertissements **1**

1.1 Encodage 1

1.2 Fonctions de hachage 2

2 Stockage de mots de passe **3**

3 Comparaison **8**

Conclusion **12**

1 Avertissements

Le TP se fait en Python3 ! Dans Jupyter, il faut choisir le noyau Python 3.

1.1 Encodage

Un caractère est stocké comme un nombre ; la façon de faire correspondre caractères et nombres s'appelle l'*encodage* (UTF8, ASCII, LATIN1…).

— `txt.encode("...")` transforme une chaîne (`str`) en **chaîne binaire** (`bytes`), reconnaissable au `b` devant ;

— `txt.decode("...")` fait l'inverse, et provoque une erreur si un octet n'existe pas dans l'encodage choisi.

En exécutant l'exemple du sujet, on obtient :

```
Il était une fois...
b'Il \xc3\xa9tait une fois...'        <- utf-8  : é = 2 octets c3 a9
b'Il \xe9tait une fois...'            <- latin1 : é = 1 octet  e9
b'Ben \xc3\xa7a alors !'
<class 'bytes'>
Ben ça alors !
UnicodeDecodeError: 'ascii' codec can't decode byte 0xc3 in position 4
```

À retenir pour la suite : les fonctions de hachage travaillent sur des `bytes`. Il faudra donc toujours écrire `mot_de_passe.encode('utf-8')` avant de hacher.

1.2 Fonctions de hachage

Rappel du cours
Une fonction de hachage h prend en entrée un bit-string de n'importe quelle taille et retourne une empreinte (*digest*) de taille **fixe** n :

$$h : \{0;1\}^* \longrightarrow \{0;1\}^n$$

Propriétés attendues :

- **résistance à la pré-image** : connaissant y, il est difficile de trouver x tel que $h(x) = y$;
- **résistance à la seconde pré-image** : connaissant x, il est difficile de trouver $x' \neq x$ tel que $h(x') = h(x)$;
- **résistance aux collisions** : il est difficile de trouver $x \neq x'$ tels que $h(x) = h(x')$.

Pour SHA-256 (famille SHA-2), $n = 256$ bits, soit 64 caractères hexadécimaux.

Avec la bibliothèque `hashlib`, on hache une chaîne binaire avec `f_hash(txt).hexdigest()` :

```
import hashlib

s2 = 'bonjour !'.encode('utf-8')
s2.hex()
hashlib.sha256(s2).hexdigest()
```

```
'626f6e6a6f75722021'
'4fdbad33e574248f84f44f3143896ca36c25b039480a7ff2e5dbabb2d6601d83'
```

Attention aux formats : `hashlib.sha256(...)` prend des `bytes` et renvoie un objet ; c'est `.hexdigest()` qui donne l'empreinte sous forme de `str` hexadécimale (64 caractères). On remarque aussi qu'en changeant un seul caractère (« bonjour » / « bonjour ! »), l'empreinte change complètement.

Exercice 1. On connaît le haché SHA-256 d'un mot de 5 lettres minuscules :

`9e69e7e29351ad837503c44a5971edebc9b7e6d8601c89c284b1b59bf37afa80`

Idée. SHA-256 est résistante à la pré-image : on ne peut pas « remonter » le haché. Mais ici l'ensemble des mots possibles est petit : $26^5 = 11\,881\,376$ mots. On peut donc tous les hacher et ranger les résultats dans une **table de hachage** (un dictionnaire Python) : `haché → mot`. C'est exactement la `Rainbow Table` du cours (« une table précalculée de valeurs en clair et leurs hash associés »).

On génère les mots avec 5 boucles imbriquées, une par lettre :

```
alphabet = "abcdefghijklmnopqrstuvwxyz"

table = {}
for a in alphabet:
    for b in alphabet:
        for c in alphabet:
            for d in alphabet:
                for e in alphabet:
                    mot = a + b + c + d + e
                    table[hashlib.sha256(mot.encode('utf-8')).hexdigest()] = mot

len(table)
```

```
11881376
```

Une fois la table construite, la recherche est immédiate :

```
table["9e69e7e29351ad837503c44a5971edebc9b7e6d8601c89c284b1b59bf37afa80"]
```

```
'qwert'
```

Le mot de passe est `qwert`. Sur ma machine, la construction prend environ 17 s et la table occupe environ 2,4 Go de mémoire. Mais elle ne se calcule qu'**une seule fois**. Ensuite, n'importe quel haché d'un mot de 5 lettres est retrouvé instantanément. C'est pour cela que le cours dit que stocker seulement le haché du mot de passe est « peu sécurisé ».

2 Stockage de mots de passe

On utilise `sqlite3` avec le code du sujet : une base `donnees.db` et une table `utilisateurs (name, password)`.

```
import sqlite3

# ouvrir la connexion ou créer la base
connection = sqlite3.connect("donnees.db")

# Créer un curseur
curseur = connection.cursor()

# Créer la table
curseur.execute("CREATE TABLE IF NOT EXISTS utilisateurs (name TEXT, password TEXT)")
```

Deux commandes à connaître :

— `curseur.execute(requete, (valeurs,))` exécute une requête SQL ; les `?` sont remplacés par les valeurs du tuple ;

— `curseur.fetchall()` renvoie le résultat sous forme de **liste de tuples**, par exemple `[('alice', 'soleil')]`. Une liste vide `[]` veut dire « aucune ligne trouvée ».

Après un `INSERT`, il faut `connection.commit()` pour enregistrer dans le fichier.

Exercice 2. Niveau de sécurité $-\infty$

> **Cours**
> identifiant + mot de passe en **clair** : Interdit !!!

La fonction `AjoutUtilisateur()` est donnée dans le sujet : elle demande un login, vérifie qu'il n'existe pas, demande deux fois le mot de passe et l'insère **tel quel** dans la base.

1. Après avoir exécuté `AjoutUtilisateur()` (ici avec le login `alice` et le mot de passe `soleil`), on affiche toute la table :

```
curseur.execute("SELECT * FROM utilisateurs")
curseur.fetchall()
```

```
[('alice', 'soleil')]
```

Le mot de passe est lisible directement : toute personne qui accède au fichier `donnees.db` (fuite de la base, administrateur…) connaît tous les mots de passe.

2. `Verification()` suit le même schéma que `AjoutUtilisateur()` : on lit le login et le mot de passe avec `input`, on récupère le mot de passe stocké pour ce login, puis on compare.

```
def Verification() :
    nom = input("login : ")
```

```
    mot_de_passe = input("Entrer votre mot de passe : ")
    curseur.execute("SELECT password FROM utilisateurs WHERE name == ?", (nom,))
    res = curseur.fetchall()
    if res == [] :
        print("Utilisateur inconnu")
    elif res[0][0] == mot_de_passe :
        print("Authentification réussie")
    else :
        print("Mot de passe incorrect")
```

`res[0][0]` : premier tuple de la liste, premier champ (le `password`).

```
login : alice   mot de passe : soleil   ->  Authentification réussie
login : alice   mot de passe : lune     ->  Mot de passe incorrect
login : carol   mot de passe : x        ->  Utilisateur inconnu
```

Remarque. Dans `AjoutUtilisateur()`, le `SELECT` est construit en collant le login dans la requête (`"... name == '" + nom + "'"`), alors que l'`INSERT` utilise des `?`. J'utilise les `?` partout, comme dans l'`INSERT` du sujet. Le contenu tapé par l'utilisateur reste ainsi une simple valeur et n'est jamais interprété comme du SQL.

Exercice 3. Niveau de sécurité -2 : on stocke `SHA256(mot_de_passe)`.

On reprend les deux fonctions avec une nouvelle table `utilisateurs_sha256`. Seule la ligne qui prépare le mot de passe change.

Attention au format (comme indiqué dans le sujet) : `sha256` attend des `bytes`, d'où `.encode('utf-8')` ; `.hexdigest()` renvoie une `str` de 64 caractères, qu'on peut stocker dans un champ `TEXT` et comparer avec `==`.

```
curseur.execute("CREATE TABLE IF NOT EXISTS utilisateurs_sha256 (name TEXT, password TEXT)")

def AjoutUtilisateur_sha256() :
    nom = input("login : ")
    curseur.execute("SELECT name FROM utilisateurs_sha256 WHERE name == ?", (nom,))
    res = curseur.fetchall()
    if (nom,) in res :
        print("Utilisateur déjà inscrit")
    else :
        mot_de_passe = input("Entrer votre mot de passe : ")
        mot_de_passe_bis = input("Entrer à nouveau votre mot de passe : ")
        if mot_de_passe == mot_de_passe_bis :
            hache = hashlib.sha256(mot_de_passe.encode('utf-8')).hexdigest()
            curseur.execute("INSERT INTO utilisateurs_sha256 (name, password) VALUES (?, ?)", (nom,
    hache))
            connection.commit()
        else :
            print("Erreur de saisie")

def Verification_sha256() :
    nom = input("login : ")
    mot_de_passe = input("Entrer votre mot de passe : ")
    curseur.execute("SELECT password FROM utilisateurs_sha256 WHERE name == ?", (nom,))
    res = curseur.fetchall()
    if res == [] :
        print("Utilisateur inconnu")
    elif res[0][0] == hashlib.sha256(mot_de_passe.encode('utf-8')).hexdigest() :
        print("Authentification réussie")
    else :
        print("Mot de passe incorrect")
```

Contenu de la table :

```
[('alice', 'e93e24c1b877a15df5e4a8cbb20f351b891b750779cf5e8fb99119a7142d89cf')]
```

Pour vérifier, on **re-hache** le mot de passe saisi et on compare les deux hachés : on n'a jamais besoin de connaître le mot de passe en clair.

Pourquoi seulement -2 **?** Deux utilisateurs avec le même mot de passe ont le même haché. Surtout, une `Rainbow Table` des mots de passe courants (le cours donne les TOP 25 : `123456`, `password`, `qwerty`…) retrouve immédiatement les mots de passe de la base, comme à l'exercice 1. Le cours le dit : c'est une « mauvaise méthode même avec un SHA2-256 ou même un SHA-3 ».

Exercice 4. Niveau de sécurité -1 : on stocke `SHA256(mot_de_passe + sel)` avec un **sel global**.

> **Cours – Salage**
> Le salage consiste à concaténer le mot de passe avec une chaîne de caractères avant le hachage. Avantage : invalider les `Rainbow Table` existantes. Malheureusement insuffisant.

Le sel est choisi une fois pour toutes et écrit en dur dans le code. Par rapport à l'exercice 3, seule la ligne de hachage change, avec une nouvelle table `utilisateurs_saltedsha256` :

```
SEL = "R3.09-SAE3.01"
curseur.execute("CREATE TABLE IF NOT EXISTS utilisateurs_saltedsha256 (name TEXT, password TEXT)")

def AjoutUtilisateur_saltedsha256() :
    nom = input("login : ")
    curseur.execute("SELECT name FROM utilisateurs_saltedsha256 WHERE name == ?", (nom,))
    res = curseur.fetchall()
    if (nom,) in res :
        print("Utilisateur déjà inscrit")
    else :
        mot_de_passe = input("Entrer votre mot de passe : ")
        mot_de_passe_bis = input("Entrer à nouveau votre mot de passe : ")
        if mot_de_passe == mot_de_passe_bis :
            hache = hashlib.sha256((mot_de_passe + SEL).encode('utf-8')).hexdigest()
            curseur.execute("INSERT INTO utilisateurs_saltedsha256 (name, password) VALUES (?, ?)",
    (nom, hache))
            connection.commit()
        else :
            print("Erreur de saisie")

def Verification_saltedsha256() :
    nom = input("login : ")
    mot_de_passe = input("Entrer votre mot de passe : ")
    curseur.execute("SELECT password FROM utilisateurs_saltedsha256 WHERE name == ?", (nom,))
    res = curseur.fetchall()
    if res == [] :
        print("Utilisateur inconnu")
    elif res[0][0] == hashlib.sha256((mot_de_passe + SEL).encode('utf-8')).hexdigest() :
        print("Authentification réussie")
    else :
        print("Mot de passe incorrect")
```

```
[('alice', '51b76a36418c5a18e2793957c7665babdb7f7f9d4a57f4c5f33e7f3910e9679c')]
```

Le haché de `soleil` n'est plus le même qu'à l'exercice 3 : une `Rainbow Table` « classique » ne sert plus à rien.

Pourquoi seulement -1 **?** (attaque vue en cours) L'attaquant crée un compte dont il connaît le mot de passe. Il retrouve ensuite le sel global par force brute, puis construit **une seule nouvelle** `Rainbow Table` avec ce sel, valable pour tous les utilisateurs.

Exercice 5. **Niveau de sécurité** 0 : on stocke `SHA256(mot_de_passe + sel_utilisateur)` avec un **sel aléatoire par utilisateur**.

> **Cours**
> `SHA1($pwd + $perUserSalt)` : salage aléatoire par utilisateur ⇒ une `Rainbow Table` par utilisateur.

Générer le sel. 16 octets = 16 × 8 = 128 bits, donc `getrandbits(128)`. Ensuite `hex()` l'écrit en hexadécimal, avec `0x` devant qu'on enlève avec `[2:]`. Mais `hex()` n'écrit pas les 0 de gauche : lors d'un essai, j'ai obtenu `0x4972387f5a503ebde7897a3277c085c`, qui ne fait que 31 caractères après `0x`. On complète donc à gauche avec des `0` jusqu'à 32 caractères :

```
from random import getrandbits

def nouveau_sel() :
    sel = hex(getrandbits(128))[2:]
    while len(sel) < 32 :
        sel = "0" + sel
    return sel
```

Stocker le sel dans une autre base. Le sujet demande une *autre base de données* : on ouvre une deuxième connexion vers `sels.db`, avec une table `sels (name, salt)`.

```
connection_sel = sqlite3.connect("sels.db")
curseur_sel = connection_sel.cursor()
curseur_sel.execute("CREATE TABLE IF NOT EXISTS sels (name TEXT, salt TEXT)")
curseur.execute("CREATE TABLE IF NOT EXISTS utilisateurs_randsaltedsha256 (name TEXT, password TEXT)
    ")

def AjoutUtilisateur_randsaltedsha256() :
    nom = input("login : ")
    curseur.execute("SELECT name FROM utilisateurs_randsaltedsha256 WHERE name == ?", (nom,))
    res = curseur.fetchall()
    if (nom,) in res :
        print("Utilisateur déjà inscrit")
    else :
        mot_de_passe = input("Entrer votre mot de passe : ")
        mot_de_passe_bis = input("Entrer à nouveau votre mot de passe : ")
        if mot_de_passe == mot_de_passe_bis :
            sel = nouveau_sel()
            hache = hashlib.sha256((mot_de_passe + sel).encode('utf-8')).hexdigest()
            curseur.execute("INSERT INTO utilisateurs_randsaltedsha256 (name, password) VALUES (?,
    ?)", (nom, hache))
            connection.commit()
            curseur_sel.execute("INSERT INTO sels (name, salt) VALUES (?, ?)", (nom, sel))
            connection_sel.commit()
        else :
            print("Erreur de saisie")

def Verification_randsaltedsha256() :
    nom = input("login : ")
    mot_de_passe = input("Entrer votre mot de passe : ")
    curseur.execute("SELECT password FROM utilisateurs_randsaltedsha256 WHERE name == ?", (nom,))
    res = curseur.fetchall()
    curseur_sel.execute("SELECT salt FROM sels WHERE name == ?", (nom,))
    res_sel = curseur_sel.fetchall()
    if res == [] or res_sel == [] :
        print("Utilisateur inconnu")
    elif res[0][0] == hashlib.sha256((mot_de_passe + res_sel[0][0]).encode('utf-8')).hexdigest() :
        print("Authentification réussie")
    else :
        print("Mot de passe incorrect")
```

```
utilisateurs_randsaltedsha256 (donnees.db) :
[('alice', '063c17fff26992d055dce8fd2c88fc3992028d1673d10c8733634b6160e617e4')]

sels (sels.db) :
[('alice', 'c083a6886d8be5279158baaa0ff74a23')]
```

Remarque sur les noms. Le sujet donne les mêmes noms `AjoutUtilisateur_saltedsha256()` et `Verification_saltedsha256()` aux exercices 4 et 5. Pour pouvoir garder les deux versions dans le même notebook, j'ai appelé celles de l'exercice 5 `..._randsaltedsha256()` (*random salted*).

Exercice 6. Niveau de sécurité 1 : `bcrypt`.

Cours

`bcrypt($pwd + $perUserSalt)`

- basé sur l'algorithme de chiffrement `Blowfish` ;
- plus lent à calculer : une attaque par force brute prend beaucoup plus de temps ;
- sel aléatoire : même si l'on retrouve le mot de passe d'un utilisateur, cela ne compromet pas les autres.

La bibliothèque `bcrypt` fournit trois fonctions :

- `bcrypt.gensalt()` : génère un sel aléatoire ;
- `bcrypt.hashpw(mdp, sel)` : hache le mot de passe (`bytes`) avec le sel et renvoie des `bytes` ;
- `bcrypt.checkpw(mdp, hache)` : renvoie `True` si `mdp` correspond au haché.

```python
import bcrypt

curseur.execute("CREATE TABLE IF NOT EXISTS utilisateurs_bcrypt (name TEXT, password TEXT)")

def AjoutUtilisateur_bcrypt() :
    nom = input("login : ")
    curseur.execute("SELECT name FROM utilisateurs_bcrypt WHERE name == ?", (nom,))
    res = curseur.fetchall()
    if (nom,) in res :
        print("Utilisateur déjà inscrit")
    else :
        mot_de_passe = input("Entrer votre mot de passe : ")
        mot_de_passe_bis = input("Entrer à nouveau votre mot de passe : ")
        if mot_de_passe == mot_de_passe_bis :
            hache = bcrypt.hashpw(mot_de_passe.encode('utf-8'), bcrypt.gensalt())
            curseur.execute("INSERT INTO utilisateurs_bcrypt (name, password) VALUES (?, ?)", (nom,
    hache.decode('utf-8')))
            connection.commit()
        else :
            print("Erreur de saisie")

def Verification_bcrypt() :
    nom = input("login : ")
    mot_de_passe = input("Entrer votre mot de passe : ")
    curseur.execute("SELECT password FROM utilisateurs_bcrypt WHERE name == ?", (nom,))
    res = curseur.fetchall()
    if res == [] :
        print("Utilisateur inconnu")
    elif bcrypt.checkpw(mot_de_passe.encode('utf-8'), res[0][0].encode('utf-8')) :
        print("Authentification réussie")
    else :
        print("Mot de passe incorrect")
```

```
[('alice', '$2b$12$g76hFixbC7ZUek/nwZC/tOzhGyZTKVDZum.1u4n/nPODYr6FI7.L.')]
```

Lecture du haché. `$2b$` indique la version de bcrypt, `12` le « coût » (le calcul est répété 2^{12} fois, c'est ce qui le rend lent). Les 22 caractères suivants sont **le sel**, et le reste est le haché. Le sel est donc stocké *dans* le haché : pas besoin d'une deuxième base, et `checkpw` le relit tout seul.

Formats. `hashpw` renvoie des `bytes` : on fait `.decode('utf-8')` pour stocker du texte, puis `.encode('utf-8')` pour le redonner à `checkpw`. On ne compare pas avec `==` : comme le sel est aléatoire, hacher deux fois le même mot de passe donne deux résultats différents. C'est `checkpw` qui refait le calcul avec le bon sel.

3 Comparaison

Exercice 7. On compare les temps de vérification des 5 types de stockage : clair, SHA-256, SHA-256 + sel global, SHA-256 + sel aléatoire, bcrypt.

Pour ne pas mélanger avec les comptes créés à la main, j'utilise une nouvelle base `comparaison.db`. Il suffit de rouvrir la connexion avant l'exercice : toutes les fonctions utilisent `connection` et `curseur`.

```
connection = sqlite3.connect("comparaison.db")
curseur = connection.cursor()
```

1. `generermdp(N, n, alph)` : pour chaque utilisateur, on construit un mot de passe de `n` caractères en tirant chaque caractère avec `choice(alph)`. Les comptes sont rangés en clair dans la table `utilisateurs`.

```
from random import choice

def generermdp(N, n, alph) :
    curseur.execute("DROP TABLE IF EXISTS utilisateurs")
    curseur.execute("CREATE TABLE utilisateurs (name TEXT, password TEXT)")
    for i in range(N) :
        nom = "user" + str(i)
        mot_de_passe = ""
        for j in range(n) :
            mot_de_passe = mot_de_passe + choice(alph)
        curseur.execute("INSERT INTO utilisateurs (name, password) VALUES (?, ?)", (nom,
    mot_de_passe))
    connection.commit()

generermdp(50, 8, "abcdefghijklmnopqrstuvwxyz0123456789")
curseur.execute("SELECT * FROM utilisateurs LIMIT 3")
curseur.fetchall()
```

```
[('user0', 'opbjdl61'), ('user1', 'zpe1jhfk'), ('user2', 'ifsy4fyy')]
```

2. Une fonction de stockage par type : elle lit toute la table `utilisateurs` et remplit une nouvelle table avec le mot de passe transformé. Le hachage est exactement celui des exercices 2 à 6.

```
def stockage_clair() :
    curseur.execute("DROP TABLE IF EXISTS stock_clair")
    curseur.execute("CREATE TABLE stock_clair (name TEXT, password TEXT)")
    curseur.execute("SELECT name, password FROM utilisateurs")
    for (nom, mot_de_passe) in curseur.fetchall() :
        curseur.execute("INSERT INTO stock_clair (name, password) VALUES (?, ?)", (nom, mot_de_passe
    ))
```

```
    connection.commit()

def stockage_sha256() :
    curseur.execute("DROP TABLE IF EXISTS stock_sha256")
    curseur.execute("CREATE TABLE stock_sha256 (name TEXT, password TEXT)")
    curseur.execute("SELECT name, password FROM utilisateurs")
    for (nom, mot_de_passe) in curseur.fetchall() :
        hache = hashlib.sha256(mot_de_passe.encode('utf-8')).hexdigest()
        curseur.execute("INSERT INTO stock_sha256 (name, password) VALUES (?, ?)", (nom, hache))
    connection.commit()

def stockage_saltedsha256() :
    curseur.execute("DROP TABLE IF EXISTS stock_saltedsha256")
    curseur.execute("CREATE TABLE stock_saltedsha256 (name TEXT, password TEXT)")
    curseur.execute("SELECT name, password FROM utilisateurs")
    for (nom, mot_de_passe) in curseur.fetchall() :
        hache = hashlib.sha256((mot_de_passe + SEL).encode('utf-8')).hexdigest()
        curseur.execute("INSERT INTO stock_saltedsha256 (name, password) VALUES (?, ?)", (nom, hache
    ))
    connection.commit()

def stockage_randsaltedsha256() :
    curseur.execute("DROP TABLE IF EXISTS stock_randsaltedsha256")
    curseur.execute("CREATE TABLE stock_randsaltedsha256 (name TEXT, password TEXT)")
    curseur_sel.execute("DROP TABLE IF EXISTS stock_sels")
    curseur_sel.execute("CREATE TABLE stock_sels (name TEXT, salt TEXT)")
    curseur.execute("SELECT name, password FROM utilisateurs")
    for (nom, mot_de_passe) in curseur.fetchall() :
        sel = nouveau_sel()
        hache = hashlib.sha256((mot_de_passe + sel).encode('utf-8')).hexdigest()
        curseur.execute("INSERT INTO stock_randsaltedsha256 (name, password) VALUES (?, ?)", (nom,
    hache))
        curseur_sel.execute("INSERT INTO stock_sels (name, salt) VALUES (?, ?)", (nom, sel))
    connection.commit()
    connection_sel.commit()

def stockage_bcrypt() :
    curseur.execute("DROP TABLE IF EXISTS stock_bcrypt")
    curseur.execute("CREATE TABLE stock_bcrypt (name TEXT, password TEXT)")
    curseur.execute("SELECT name, password FROM utilisateurs")
    for (nom, mot_de_passe) in curseur.fetchall() :
        hache = bcrypt.hashpw(mot_de_passe.encode('utf-8'), bcrypt.gensalt())
        curseur.execute("INSERT INTO stock_bcrypt (name, password) VALUES (?, ?)", (nom, hache.
    decode('utf-8')))
    connection.commit()
```

3. Authentification « sur la seule donnée du login » : on va chercher le mot de passe en clair dans `utilisateurs` (il joue le rôle de ce que l'utilisateur taperait), puis on fait la même vérification qu'aux exercices précédents. Les fonctions renvoient `True` ou `False`.

```
def mdp_clair(nom) :
    curseur.execute("SELECT password FROM utilisateurs WHERE name == ?", (nom,))
    return curseur.fetchall()[0][0]

def auth_clair(nom) :
    mot_de_passe = mdp_clair(nom)
    curseur.execute("SELECT password FROM stock_clair WHERE name == ?", (nom,))
    return curseur.fetchall()[0][0] == mot_de_passe

def auth_sha256(nom) :
    mot_de_passe = mdp_clair(nom)
    curseur.execute("SELECT password FROM stock_sha256 WHERE name == ?", (nom,))
    return curseur.fetchall()[0][0] == hashlib.sha256(mot_de_passe.encode('utf-8')).hexdigest()
```

```python
def auth_saltedsha256(nom) :
    mot_de_passe = mdp_clair(nom)
    curseur.execute("SELECT password FROM stock_saltedsha256 WHERE name == ?", (nom,))
    return curseur.fetchall()[0][0] == hashlib.sha256((mot_de_passe + SEL).encode('utf-8')).
    hexdigest()

def auth_randsaltedsha256(nom) :
    mot_de_passe = mdp_clair(nom)
    curseur_sel.execute("SELECT salt FROM stock_sels WHERE name == ?", (nom,))
    sel = curseur_sel.fetchall()[0][0]
    curseur.execute("SELECT password FROM stock_randsaltedsha256 WHERE name == ?", (nom,))
    return curseur.fetchall()[0][0] == hashlib.sha256((mot_de_passe + sel).encode('utf-8')).
    hexdigest()

def auth_bcrypt(nom) :
    mot_de_passe = mdp_clair(nom)
    curseur.execute("SELECT password FROM stock_bcrypt WHERE name == ?", (nom,))
    return bcrypt.checkpw(mot_de_passe.encode('utf-8'), curseur.fetchall()[0][0].encode('utf-8'))
```

4. Pour mesurer le temps, on utilise `time.time()` (le nombre de secondes écoulées) avant et après la boucle sur tous les utilisateurs. La fonction d'authentification est passée en paramètre, ce qui permet de chronométrer les 5 types avec le même code :

```python
import time

def temps_authentification(auth) :
    curseur.execute("SELECT name FROM utilisateurs")
    noms = curseur.fetchall()
    debut = time.time()
    for (nom,) in noms :
        if not auth(nom) :
            print("Échec pour", nom)
    return time.time() - debut

stockage_clair()
print("clair :", temps_authentification(auth_clair))
# ... de même pour sha256, saltedsha256, randsaltedsha256 et bcrypt
```

Résultats pour $N = 50$ utilisateurs (mots de passe de 8 caractères) :

Stockage	**Niveau**	**Temps pour 50 authentifications**
en clair	$-\infty$	0,0015 s
SHA-256	-2	0,0014 s
SHA-256 + sel global	-1	0,0015 s
SHA-256 + sel aléatoire	0	0,0022 s
bcrypt	1	**10,1 s**

Interprétation.

— Les quatre premiers modes prennent tous un temps négligeable : SHA-256 est *conçu* pour être rapide. Le sel aléatoire ajoute seulement une lecture dans la base des sels.

— bcrypt est environ **5000 fois plus lent**, soit environ 0,2 s par vérification. On retrouve l'ordre de grandeur du sujet (« de l'ordre de 10 secondes pour 50 utilisateurs »).

— Pour un utilisateur honnête, 0,2 s à la connexion ne se remarque pas. Pour un attaquant qui doit tester des millions de mots de passe, c'est énorme : avec SHA-256, la table de l'exercice 1 se construit en 17 s ; avec bcrypt, il faudrait environ $11\,881\,376 \times 0{,}2 \approx 2{,}4$ millions de secondes,

soit environ **27 jours**, et cela *pour un seul utilisateur*, puisque chaque sel est différent. C'est exactement l'argument du cours : « plus lent à calculer : procédé de brute-force beaucoup plus long ».

Bonus : Niveau de sécurité 2

> **Cours**
> `AES(bcrypt($perUserSalt, $pwd), $secretKey)`

On chiffre le haché bcrypt avec AES en mode CTR (vu au TP4) et la clé `YELLOW SUBMARINE`. Même si la base fuite, les hachés sont inutilisables sans la clé secrète, qui n'est pas stockée dans la base.

En mode CTR, le chiffrement tire un `nonce` aléatoire (`cipher.nonce`). Pour déchiffrer, il faut le même nonce : on le stocke dans le champ supplémentaire demandé. Comme le résultat d'`encrypt` est en `bytes`, on le stocke en hexadécimal avec `.hex()` et on le relit avec `bytes.fromhex(...)`.

L'exercice 7 a rouvert la connexion vers `comparaison.db` : on revient d'abord sur `donnees.db`.

```
from Crypto.Cipher import AES

connection = sqlite3.connect("donnees.db")
curseur = connection.cursor()

CLE = b"YELLOW SUBMARINE"
curseur.execute("CREATE TABLE IF NOT EXISTS utilisateurs_aes (name TEXT, password TEXT, nonce TEXT)"
    )

def AjoutUtilisateur_aes() :
    nom = input("login : ")
    curseur.execute("SELECT name FROM utilisateurs_aes WHERE name == ?", (nom,))
    res = curseur.fetchall()
    if (nom,) in res :
        print("Utilisateur déjà inscrit")
    else :
        mot_de_passe = input("Entrer votre mot de passe : ")
        mot_de_passe_bis = input("Entrer à nouveau votre mot de passe : ")
        if mot_de_passe == mot_de_passe_bis :
            hache = bcrypt.hashpw(mot_de_passe.encode('utf-8'), bcrypt.gensalt())
            cipher = AES.new(CLE, AES.MODE_CTR)
            chiffre = cipher.encrypt(hache)
            curseur.execute("INSERT INTO utilisateurs_aes (name, password, nonce) VALUES (?, ?, ?)",
                            (nom, chiffre.hex(), cipher.nonce.hex()))
            connection.commit()
        else :
            print("Erreur de saisie")

def Verification_aes() :
    nom = input("login : ")
    mot_de_passe = input("Entrer votre mot de passe : ")
    curseur.execute("SELECT password, nonce FROM utilisateurs_aes WHERE name == ?", (nom,))
    res = curseur.fetchall()
    if res == [] :
        print("Utilisateur inconnu")
    else :
        chiffre = bytes.fromhex(res[0][0])
        nonce = bytes.fromhex(res[0][1])
        cipher = AES.new(CLE, AES.MODE_CTR, nonce=nonce)
        hache = cipher.decrypt(chiffre)
        if bcrypt.checkpw(mot_de_passe.encode('utf-8'), hache) :
            print("Authentification réussie")
        else :
            print("Mot de passe incorrect")
```

```
[('alice',
  'c733974200cd0299ede4e91790c343d9f19dcd19e893cbd3f93d2b12ec414613aec34a1546973155
   5d6f110d81717fc744be8d056ae31136207e3965',
  '9e64a292bf41a020')]
```

Le champ `password` ne ressemble plus du tout à un haché bcrypt (il n'y a plus de `$2b$12$` visible) ; le dernier champ est le nonce (8 octets, soit 16 caractères hexadécimaux).

Remarque sur la clé. `YELLOW SUBMARINE` fait 16 caractères, donc 16 octets = 128 bits : avec cette clé, `AES.new` fait en réalité de l'**AES-128**. Pour faire de l'AES-256 comme l'annonce le sujet, il faudrait une clé de 32 octets. J'ai gardé la clé imposée par le sujet.

Conclusion

On retrouve dans le TP les niveaux de stockage de la conclusion du cours :

Niveau	**Stockage**	**Exercice**	**Faiblesse / apport**
$-\infty$	en clair	2	interdit : tout est lisible
-2	Haché(pwd)	3	Rainbow Tables (exercice 1)
-1	Haché(pwd + Salt)	4	un seul sel à retrouver pour tous
0	Haché(pwd + Salt-user)	5	une Rainbow Table par utilisateur
1	bcrypt(pwd + Salt-user)	6	hachage lent (exercice 7)
2	AES(bcrypt(…), SecretKey)	Bonus	base inutilisable sans la clé